

System Requirements Document

Course Feedback App

Tyrone Chandler

4/29/26

Table of Contents

1. Customer Problem Statement and System Requirements
2. Functional Requirement Specification
3. System Sequence Diagram
4. Activity Diagram
5. User Interface Specification
6. Traceability Matrix
7. System Architecture and System Design
8. Algorithms and Data Structures
9. User Interface Design and Implementation
10. Design of Tests
11. Project Plan

1. Customer Problem Statement and System Requirements

Customer Problem Statement

Usually both students and teachers want a fast and simple way to get their course feedback without having to use a large survey tool or inputting the responses into a spreadsheet by hand. Manual feedback collection can be very time consuming, quite inconsistent, and a real headache when it comes to summarizing. The Course Feedback App is designed to solve this problem by giving users a small desktop application through which they can enter course feedback, review the feedback already submitted, see the automatically calculated average rating, and export the results to a CSV file that can be easily opened in Excel.

System Requirements

Requirement ID	Requirement Description
SR-1	The system shall allow the user to enter a course name.
SR-2	The system shall allow the user to select a rating from 1 to 5.
SR-3	The system shall allow the user to enter a written comment.
SR-4	The system shall validate that the course name and comment are not blank.
SR-5	The system shall store each feedback entry during the session.
SR-6	The system shall display submitted feedback in the output area.
SR-7	The system shall automatically calculate the average rating.
SR-8	The system shall export feedback entries to a CSV file.
SR-9	The system shall display user friendly error messages.
SR-10	The system shall be simple enough to run in Eclipse using Java Swing.

2. Functional Requirement Specification

Actors and Goals

Actor	Goal
Student/User	Enter course feedback, view submitted feedback, calculate the average rating, and export results.
Instructor/Reviewer	Use exported CSV feedback results to review course feedback in a spreadsheet.

System	Validate input, store feedback, calculate the average rating, and generate the CSV file.
--------	--

Functional Requirements

ID	Function	Description
FR-1	Add Feedback	The user enters a course name, selects a rating, and enters a comment. The system validates the fields and stores the feedback.
FR-2	View Feedback	The system displays each submitted feedback entry in the output area.
FR-3	Calculate Average Rating	The system adds all rating values and divides the total by the number of entries.
FR-4	Export Results	The system writes course name, rating, and comment data to a CSV file.
FR-5	Error Handling	The system shows a message if required fields are missing or if the export fails.

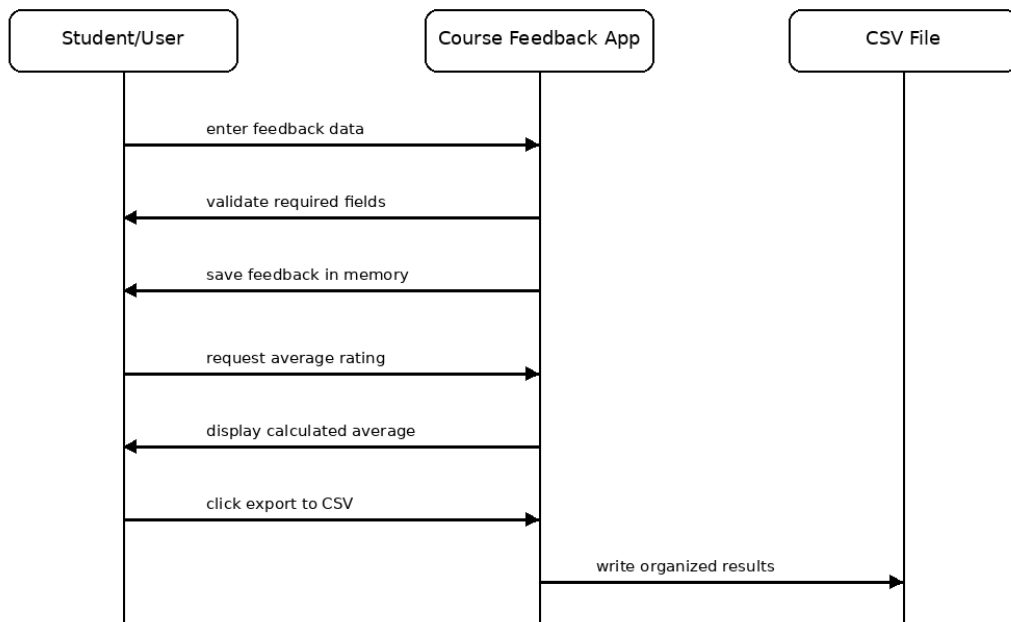
Use Case Descriptions

Use Case ID	Use Case Name	Brief Description
UC-1	Add Course Feedback	User enters course name, rating, and comment. System saves the feedback and displays it.
UC-2	Show Average Rating	User clicks Show Average Rating. System calculates and displays the average.
UC-3	Export to CSV	User clicks Export to CSV. System creates course_feedback_results.csv.

3. System Sequence Diagram

The system sequence diagram shows how the user interacts with the Course Feedback App and how the app writes exported feedback to a CSV file.

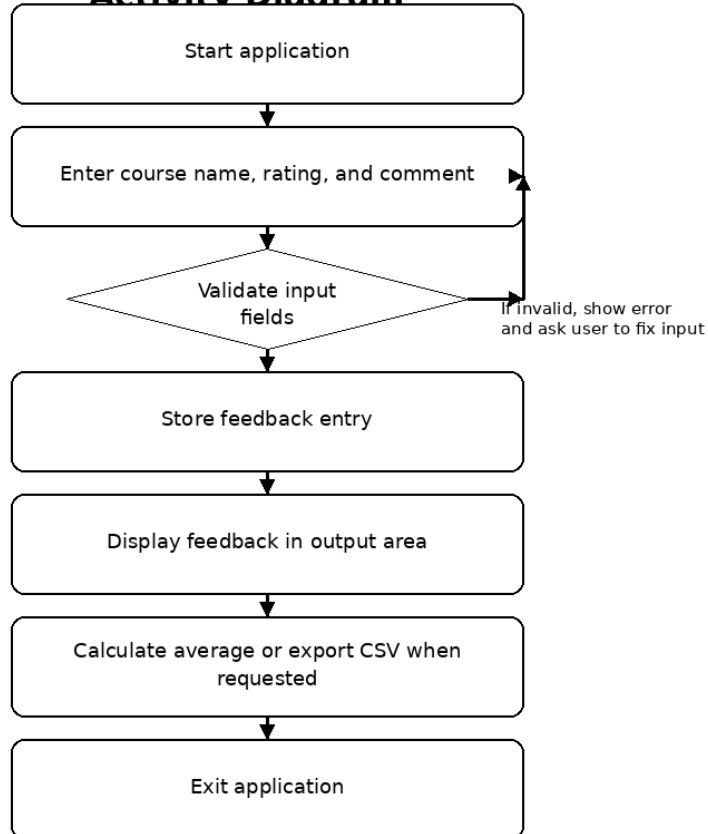
System Sequence Diagram



4. Activity Diagram

The activity diagram shows the main workflow from launching the application to entering feedback, validating input, saving the entry, calculating the average, exporting results, and exiting.

Activity Diagram



5. User Interface Specification

Interface Overview

The user interface is a simple Java Swing window. It uses plain labels, a course name text field, a rating drop down menu, a comment box, and three buttons. This design is easy to understand and avoids unnecessary screens or login steps.

User Interface Wireframe

Course Feedback App

Course Name:

Rating:

1 2 3 4 5

Comment:

Add Feedback

Show Average Rating

Export to CSV

Output Area:

Interface Elements

UI Element	Purpose
Course Name Field	Accepts the name of the course receiving feedback.
Rating Drop Down	Allows the user to choose a number from 1 to 5.
Comment Box	Allows the user to enter written feedback.
Add Feedback Button	Stores and displays the feedback entry.
Show Average Rating Button	Automatically calculates the average rating.
Export to CSV Button	Automatically saves feedback results in a CSV file.
Output Area	Displays submitted feedback entries.

6. Traceability Matrix

The traceability matrix connects system requirements to features, design elements, and test cases. This helps show that the project requirements are covered by the implementation and testing plan.

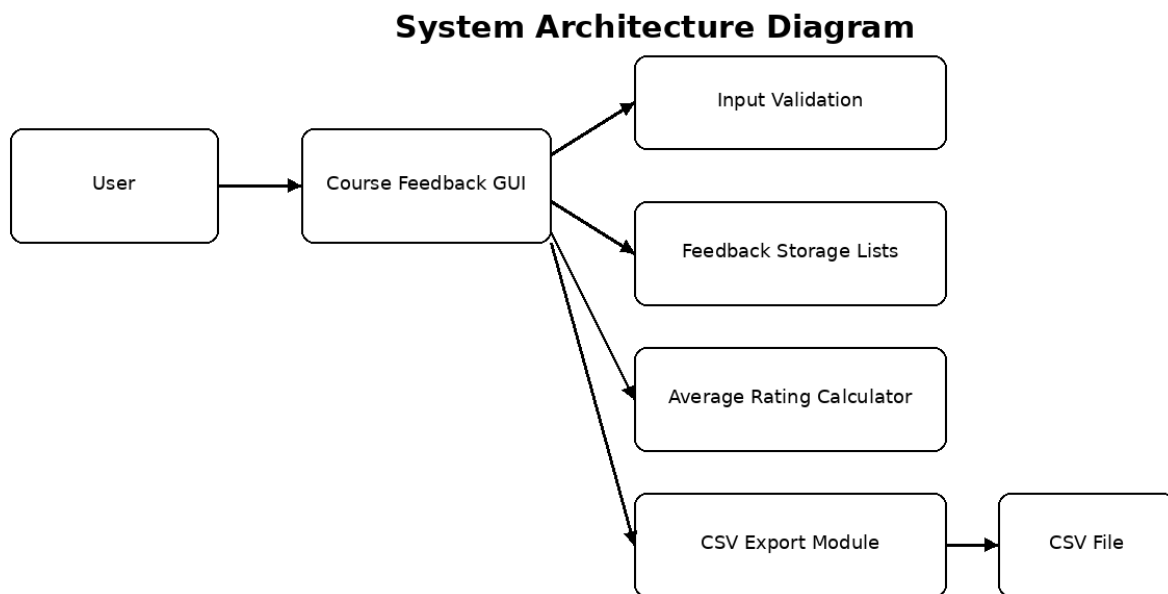
Requirement	Functional Requirement	Design/Implementation Element	Test Case
-------------	------------------------	-------------------------------	-----------

SR-1, SR-2, SR-3	FR-1 Add Feedback	Course name field, rating drop down, comment box	TC-1
SR-4, SR-9	FR-5 Error Handling	Validation messages	TC-2
SR-5, SR-6	FR-2 View Feedback	Output area and feedback lists	TC-3
SR-7	FR-3 Calculate Average Rating	Average rating calculation	TC-4
SR-8	FR-4 Export Results	CSV export module	TC-5
SR-10	All functions	Java Swing desktop app	TC-6

7. System Architecture and System Design

Architecture Description

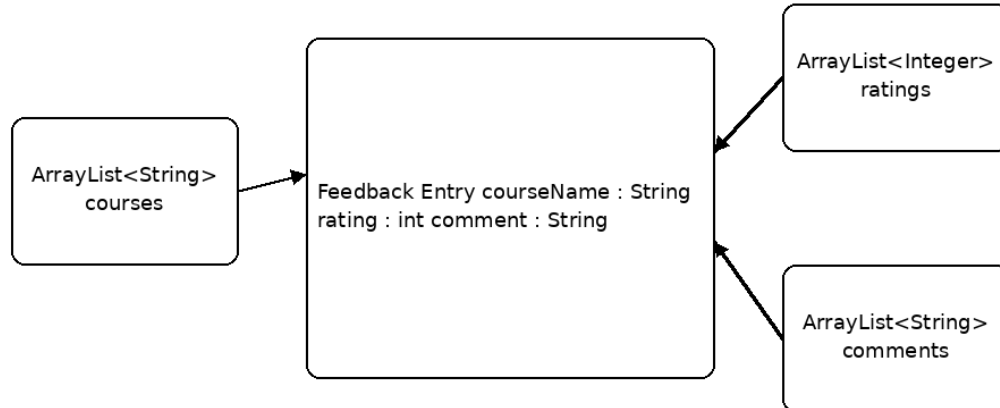
The Course Feedback App uses a simple desktop architecture. The user interacts with a Java Swing interface. The interface sends data to internal lists that store course names, ratings, and comments. The system can calculate the average rating from the stored ratings and can write all stored data to a CSV file.



Data Model Implementation

The data model is intentionally simple. The app stores course names, ratings, and comments in ArrayList structures. Each index position represents one feedback entry. For example, `courses[0]`, `ratings[0]`, and `comments[0]` belong to the same feedback record.

Data Model Diagram



Data Item	Type	Purpose
courses	ArrayList<String>	Stores course names.
ratings	ArrayList<Integer>	Stores rating values from 1 to 5.
comments	ArrayList<String>	Stores written feedback comments.
course_feedback_results.csv	CSV file	Stores exported feedback results for Excel.

Design Decisions

Java Swing was selected because it is included with standard Java and works well in Eclipse without extra setup. ArrayLists were selected because the project is small and the data only needs to be stored during the session before export. CSV was selected because it is easy to open in Excel and demonstrates automatic data organization.

8. Algorithms and Data Structures

Algorithms

The main algorithm used in the system is the average rating calculation. The system loops through all rating values, adds them together, and divides the total by the number of feedback entries. This allows the app to automatically summarize feedback instead of making the user calculate it manually.

Average Rating = Total of All Ratings / Number of Feedback Entries

CSV Export Algorithm

The export algorithm opens a file named `course_feedback_results.csv`, writes a header row, loops through all saved feedback entries, writes each entry as a CSV row, and then closes the file. This turns the feedback list into a spreadsheet ready format automatically.

Data Structures

The system uses ArrayLists because they are simple, flexible, and easy to use for a small project. They allow the program to add new feedback entries as the user enters them. The project does not need a database because the assignment can be demonstrated with local session data and a CSV export file.

9. User Interface Design and Implementation

Implementation Summary

The interface is implemented in one Java class named `CourseFeedbackApp`. The class creates the main JFrame window, input fields, buttons, and output area. Button actions are connected to methods that add feedback, calculate the average rating, and export data. This keeps the project easy to read and easy to demonstrate.

Coding Standards and Practices

Practice	How It Is Used
Naming	Variables and methods use clear names such as <code>courseField</code> , <code>ratingBox</code> , <code>addFeedback</code> , <code>showAverage</code> , and <code>exportToCSV</code> .
Comments	Comments can be added to explain the purpose of major sections without making the code cluttered.
Modularity	The main actions are separated into methods instead of placing all logic inside the main method.
Error Handling	The app checks for missing input and handles file export errors with messages.
Maintainability	The project is small and readable, so future features can be added without rewriting the entire app.

Deployment Readiness

The app can be run directly in Eclipse. For final delivery, the source code can be uploaded to GitHub with a README file, screenshots, the brochure, the demo link, and this documentation. The project can later be exported as a runnable JAR file if needed.

10. Design of Tests

The testing plan includes unit, integration, and system testing. Since this is a small Java Swing project, testing can be performed manually and with simple method level checks. The goal is to confirm that the app works correctly and handles common mistakes.

Test Cases

Test Case ID	Test Name	Input/Action	Expected Result
TC-1	Add valid feedback	Course = I101, Rating = 5, Comment = Good course	Feedback is saved and displayed.
TC-2	Blank field validation	Leave course name or comment blank	System shows an error message and does not save entry.
TC-3	View displayed entries	Add two feedback entries	Both entries appear in the output area.
TC-4	Average rating calculation	Ratings 5 and 3	System shows average rating of 4.00/5.
TC-5	CSV export	Add feedback and click Export to CSV	CSV file is created with header and feedback rows.
TC-6	Full system workflow	Add feedback, calculate average, export CSV	All core features work together correctly.

Performance Optimization

The system uses simple loops and ArrayLists, which are efficient for the small amount of feedback data expected in this project. The rating average calculation runs quickly because it only loops through the list once. CSV export is also efficient because it writes each feedback entry one time.